
Intermediate Code

- Memperkecil usaha dalam membuat compiler dari sejumlah bahasa ke sejumlah mesin
 - Lebih *Machine Independent*, hasil dari intermediate code dapat digunakan lagi pada mesin lainnya
 - Proses Optimasi lebih mudah. Lebih mudah dilakukan pada intermediate code dari pada *program sumber* (source program) atau pada kode *assembly* dan kode mesin
 - Intermediate code ini lebih mudah dipahami dari pada *kode assembly* atau kode mesin
 - Kerugiannya adalah melakukan 2 kali transisi, maka dibutuhkan waktu yang relatif lama
-

Intermediate Code

Ada dua macam intermediate code yaitu **Notasi Postfix** dan **N-Tuple**

Notasi **POSTFIX**

<Operand> <Operand> < Operator>

Misalnya :

$(a + b) * (c + d)$

maka Notasi postfixnya

$ab+ cd+ *$

Semua instruksi kontrol program yang ada diubah menjadi notasi postfix, misalnya

IF <expr> **THEN** <stmt1> **ELSE** <stmt2>

POSTFIX

Diubah ke postfix menjadi ;

<expr> <label1> **BZ** <stmt1> <label2> **BR** < stmt2>

BZ : Branch if zero (salah)

BR: melompat tanpa harus ada kondisi yang dites

Contoh : **IF** a > b **THEN** c := d **ELSE** c := e

POSTFIX

Contoh : **IF** $a > b$ **THEN** $c := d$ **ELSE** $c := e$

Dalam bentuk Postfix

11 a	19
12 b	20 25
13 >	21 BR
14 22	22 c
15 BZ	23 e
16 c	24 :=
17 d	25
18 :=	

bila expresi $(a > b)$ salah, maa loncat ke instruksi 22, Bila expresi $(a > b)$
benar tidak ada loncatan, instruksi berlanjut ke 16-18 lalu loncat ke 25

POSTFIX

Contoh:

WHILE <expr> **DO** <stmt>

Diubah ke postfix menjadi ; <expr> <label1> **BZ** <stmt> <label2> **BR**

Instruksi : a:= 1
 WHILE a < 5 DO
 a := a + 1

Dalam bentuk Postfix

10 a	18 a
11 1	19 a
12 :=	20 1
13 a	21 +
14 5	22 :=
15 <	23 13
16 25	24 BR
17 BZ	25

TRIPLES NOTATION

Notasi pada triple dengan format

<operator> <operand> <operand>

Contoh:

$A := D * C + B / E$

Jika dibuat intermediate code triple:

1. $*$, D, C
2. $/$, B, E
3. $+$, (1), (2)
4. $:=$, A, (3)

Perlu diperhatikan presedensi (hirarki) dari operator, operator perkalian dan pembagian mendapatkan prioritas lebih dahulu dari oada penjumlahan dan pengurangan

TRIPLES NOTATION

Contoh lain:

IF $X > Y$ **THEN**

$X := a - b$

ELSE

$X := a + b$

Intermediate code triple:

1. $>, X, Y$
2. BZ, (1), (6) bila kondisi 1 loncat ke lokasi 6
3. $-, a, b$
4. $:=, X, (3)$
5. BR, , (8)
6. $+, a, b$
7. $:=, X, (6)$

TRIPLES NOTATION

Kelemahan dari notasi **triple** adalah sulit pada saat melakukan optimasi, maka dikembangkan **Indirect triples** yang memiliki dua list; list instruksi dan list eksekusi. List Instruksi berisikan notasi triple, sedangkan list eksekusi mengatur eksekusinya; contoh

A := B + C * D / E

F := C * D

List Instruksi

1. *, C, D
2. /, (1), E
3. +, B, (2)
4. :=, A, (3)
5. :=, F, (1)

List Eksekusi

1. 1
2. 2
3. 3
4. 4
5. 1
6. 5

Quadruples Notation

Format dari quadruples adalah

<operator> <operand> <operand> <result>

Result atau hasil adalah *temporary variable* yang dapat ditempatkan pada *memory* atau *register* . Problemnya adalah bagaimana mengelola temporary variable seminimal mungkin

Contoh:

$A := D * C + B / E$

Jika dibuat intermediate codenya :

1. *, D, C, T1
2. /, B, E, T2
3. +, T1, T2, A

Quadruples Notation

- Hasil dari tahapan analisis diterima oleh code generator (pembangkit kode)
- Intermediate code ditransformasikan kedalam bahasa assembly atau mesin
- Misalnya

(A+B)*(C+D) dan diterjemahkan kedalam bentuk quadruple:

1. +, A, B, T1
2. +, C, D, T2
3. *, T1, T2, T3

Dapat ditranslasikan kedalam bahasa assembly dengan accumulator tunggal:

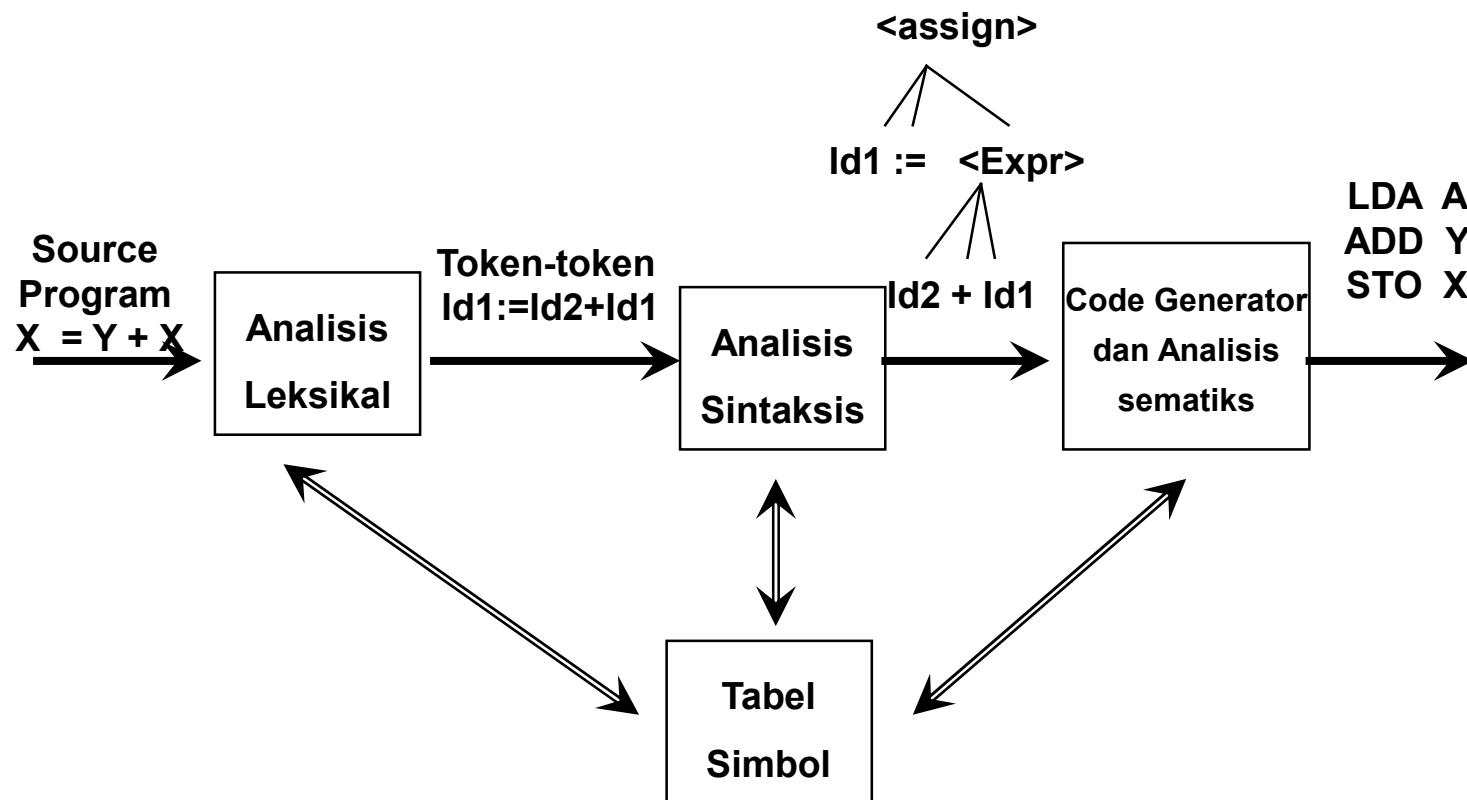
Code Generator

LDA	A	(isi A ke dalam accumulator)
ADD	B	(isi accumulator dijumlahkan dengan B)
STO	T1	(Simpan isi Accumulator ke T1)
LDA	C	
ADD	D	
STO	T2	
LDA	T1	
MUL	T2	
STO	T3	

hasil dari *code generator* akan diterima oleh *code optimization* , Misalnya untuk kode assembly diatas dioptimasikan menjadi:

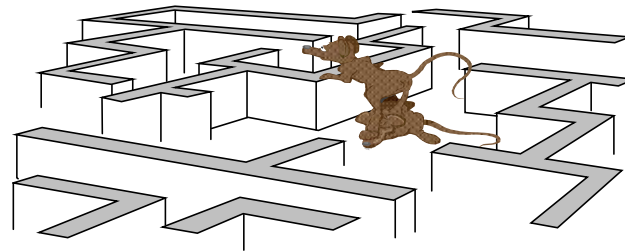
LDA	A
ADD	B
STO	T1
LDA	C
ADD	D
MUL	T1
STO	T2

Perjalanan sebuah intruksi



Error Handling

- Kesalahan Program
- Penanganan Kesalahan
- Reaksi Compiler Pada kesalahan
- Error Recovery
- Error repair



Kesalahan Program

Kesalahan Program dapat
berupa

- ✓ Kesalahan **leksikal**
- ✓ Kesalahan **Sintaks**



emantics

Error Handling - Kesalahan Program

Kesalahan Program dapat berupa

- ✓ Kesalahan **leksikal**
 - Kesalahan dalam mengetik/mengeja
 - Misal **THEN** dituliskan dengan **TEN** atau **THN**

 - ✓ Kesalahan **Sintaks**
 - misalnya dalam operasi aritmatika dengan tanda kurung yang jumlahnya kurang, contoh
 - $A := X + (B * (C + D))$

 - ✓ Kesalahan Semantics
-

Error Handling - Kesalahan Program

- ✓ Kesalahan **Semantics**

- *Tipe data yang salah*

Contoh : `int c;`

`c = 1.5 * 0.78`

- *Variable belum didefinisikan*

Misal : `B := B + 1`

tetapi `b` belum didefinisikan

Error Handling - Penanganan Kesalahan

Langkah-langkah:

- ✓ Mendeteksi kesalahan
- ✓ Melaporkan kesalahan
- ✓ Tindak lanjut perbaikan



Error Handling - Penanganan Kesalahan

- ✓ Misal: compiler menemukan kesalahan, yang bisa meliputi
 - *Kode kesalahan*
 - *Pesan Kesalahan dalam bahasa alami*
 - *Nama dan atribut identifier*
 - contoh : error 162 Jumlah: Unknow identifier
 - Dapat diartikan: Kode kesalahan =162, pesan kesalahan = *unknown identifier*, nama *identifier* = jumlah
-

Error Handling - Reaksi terhadap Kesalahan

Ada Beberapa reaksi yang dilakukan oleh compiler

- ✓ *Reaksi-reaksi yang tidak dapat diterima*
 - ✓ *Reaksi yang benar, tapi kurang dapat diterima dan kurang bermanfaat*
-

Error Handling - Reaksi terhadap Kesalahan

Ada Beberapa reaksi yang dilakukan oleh compiler

✓ ***Reaksi-reaksi yang tidak dapat diterima***

- *Compiler crash*: Berhenti atau hang
 - *Looping* : compiler tidak bisa berhenti (infinite/unbounded loop)
 - Menghasilkan Obyek program yang salah : berbahaya, bisa diketahui/muncul setelah program dieksekusi
-

Error Handling - Reaksi terhadap Kesalahan

Ada Beberapa reaksi yang dilakukan oleh compiler

✓ ***Reaksi yang benar, tapi kurang dapat diterima dan kurang bermanfaat***

- Compiler menemukan kesalahan pertama, melaporkannya, lalu berhenti (halt)
 - Pemrogram membuang waktu untuk melakukan pengulangan kompilasi untuk setiap kali terdapat sebuah error
-

Error Handling - Reaksi terhadap Kesalahan

✓ Reaksi-reaksi yang dapat diterima

- Reaksi yang sudah dapat dilakukan ; *Compiler* melaporkan Error
 - Recovery : Pemulihan
 - Repair : Perbaikan
 - Reaksi yang belum dapat dilakukan
 - Compiler mengkoreksi kesalahan
 - Menghasilkan obyek program sesuai yang diinginkan pemrogram
 - Compiler memiliki kemampuan untuk 'mengetahui' maksud dari pemrogram
 - Belum diimplementasikan pada program (sekarang ini)
-

Error Handling - Error Recovery

Bertujuan mengembalikan *parser* ke kondisi stabil agar supaya dapat melanjutkan proses *parsing* ke posisi selanjutnya.

✓ ***Mekanisme Ad Hoc***

- Recovery yang dilakukan tergantung dari si pembuat compiler
- Tidak terikat pada suatu aturan tertentu
- Disebut juga dengan istilah *purpose error recovery*

✓ ***Syntax directed Recovery***

misal begin

A := A + 1 ;

B := B + 1;

C := C + 1

end ;

Error Handling - Error Recovery

Pada contoh diatas, compiler akan mengenali sebagai (dalam Notasi BNF)

begin <statement> ? , statement> ; <statement> end;

? Akan diperlakukan sebagai ';'

- **Second Error Recovery** : untuk melokalisir kesalahan

- **Panic Mode**

- Maju terus sampai ketemu delimiter
 - Contoh : IF A = 1 Kondisi := true;
 - Pada kondisi diatas **THEN** tidak ada, compiler melanjutkan sampai ketemu delimiter (;)

- **Unit Deletion**

- Menghapus keseluruhan suatu unit sintaksik (misalnya : <block>, <exp>, <statement> dan sebagainya
 - Mempermudah untuk melakukan error repairing
-

Error Handling - Error Recovery

- **Context Sensitive Recovery**
 - Berkaitan dengan semantics
 - contoh : B := 'Budi Luhur'
 - Pada awal program variabel B belum dideklarasikan, maka berdasarkan permunculannya maka diasumsikan variabel B bertipe *string*
-

Error Handling - Error repair

Memperbaiki kesalahan dan membuat source program valid (memodifikasi)

- ✓ Mekanisme Ad Hoc
 - Tergantung pada sipembuat compiler
- ✓ Syntax directed Repair
 - Menyisipkan / membuang simbol terminal yang dianggap hilang atau yang menyebabkan error
 - contoh **WHILE** A < 1
I := I + 1;
 - compiler akan menyisipkan **DO**



Error Handling - Error repair

- Contoh lain

Procedure Increment ;

begin

$x := X + 1$

end;

end;



- Kelebihan simbol **end**, yang menyebabkan kesalahan, maka compiler akan membuangnya

Error Handling - Error repair

✓ Context Sensitive Repair

- Tipe identifier: membuat *identifier dummy*

```
var A : String  
begin  
  A := 0;  
end
```

maka compiler akan memperbaiki kesalahan dengan membuat *identifier baru* ,
misalnya B bertipe integer

- Spelling Repair: memperbaiki kesalahan pengetikan pada identifier, misalnya:

```
WHILLE A = 1 DO
```

identifier yang salah tersebut diperbaiki menjadi **WHILE**

Teknik Optimasi

- **Dependensi Optimasi**
- **Optimasi Lokal**
- **Optimasi Global**

- ***Dependensi Optimasi***

bertujuan untuk menghasilkan kode program yang berukuran lebih kecil dan lebih cepat

- Machine Dependent Optimizer
 - Machine Independent Optimizer (Optimasi lokal dan Optimasi global)
-

Teknik Optimasi : Optimasi Lokal

Optimasi Lokal : adalah optimasi yang dilakukan hanya pada suatu blok dari source code, dengan cara:

- ✓ ***Folding***

menganti konstanta atau ekspresi yang bisa dievaluasi pada saat *compile time* dengan nilai komputasinya. Misalnya:

$A := 2 + 3 + B$ bisa diganti dengan $A := 5 + B$

5 dapat menggantikan ekspresi $2 + 3$

- ✓ ***Redundant-Subexpression Elimination***

hasilnya digunakan lagi dari pada dilakukan komputasi ulang, contoh:

$A := B + C$

$X := Y + B + C$

Teknik Optimasi : Optimasi Lokal

- ✓ **Optimasi dalam sebuah Iterasi**
 - ***Loop Unrolling***: Menganti suatu *loop* dengan menulis statement yang ada dalam loop ditulis beberapa kali
 - Karena sebuah iterasi pada implementasi ke level rendah, memerlukan :
 - Inisialisasi nilai awal, pada loop dilakukan sekali pada saat permulaan eksekusi loop
 - Penge-test-an, apakah variabel loop telah mencapai kondisi terminasi
 - Adjustment yaitu: penambahan atau pengurangan nilai pada variabel loop dengan jumlah tertentu
 - Operasi yang terjadi pada tubuh perulangan (loop body)
-

Teknik Optimasi : Optimasi Lokal

- Contoh :

FOR I := 1 to 2 DO

A[I] := 0;

dapat dioptimaskan menjadi

A[1] := 0;

A[2] := 0;

- Frequency Reduction: Pemindahan statement ke tempat yang lebih jarang dieksekusi, contoh

FOR I:= 1 to 10 DO

BEGIN

X := 5

A := A + 1

END:



X := 5

FOR I:= 1 to 10 DO

BEGIN

A := A + 1

END:

Teknik Optimasi : Optimasi Lokal

- ✓ Strength Reduction

- Penggantian suatu operasi dengan operasi lain yang lebih cepat dieksekusi
- misalnya: pada komputer operasi perkalian memerlukan waktu eksekusi lebih banyak dari pada operasi penjumlahan
- contoh lain

$A := A + 1$

- dapat digantikan dengan

INC(A)

Teknik Optimasi : Optimasi GLobal

Optimasi global biasanya dilakukan dengan suatu graph terarah yang menunjukkan jalur yang mungkin selama eksekusi program

ada dua kegunaan yaitu bagi programmer dan compiler itu sendiri

✓ Bagi Programmer

- *Unreachable/dead code*: Kode yang tidak pernah dieksekusi

- misalnya :

```
X := 5;
```

```
IF X = 0 THEN
```

```
    A := A + 1
```

Instruksi

A := A + 1 tidak pernah dikerjakan

Teknik Optimasi : Optimasi GLobal

- ❑ ***Unused parameter*** : parameter yang tidak pernah digunakan dalam procedure
 - Misalnya :

```
procedure penjumlahan(a,b,c ; Integer);  
    var x : integer;  
    begin  
        x := a + b;  
    end
```

Parameter **c** tidak pernah digunakan sehingga tidak perlu diikuti sertakan

Teknik Optimasi : Optimasi GLobal

- ***Unused Variabel*** : variabel yang yang tidak pernah dipergunakan

```
Program pendek;  
var a, b: integer  
begin  
    a := 5;  
end;
```

- B tidak pernah digunakan
-

Teknik Optimasi : Optimasi GLobal

- **Variable** : variabel yang dipakai tanpa nilai awal. Contoh
Program Awal;
var a, b: integer
begin
 a := 5
 a := a + b;
end;
 - **variabel b** digunakan tetapi tidak memiliki harga awal
 - ✓ **Bagi Compiler**
 - Meningkatkan efisiensi eksekusi program
 - Menghilangkan useless code/kode yang tidak terpakai
-

```
var A, B, C, D, E, I, J, X, Y : integer;
begin
    B := 5;
    A := 10 - B / 4 * 3 + 2;
    C := A + B;
    Y := 10;
    D := A + B - E;
    for I := 1 to 85 do
        begin
            X := X + 1;
            B := B - X;
            Y := 7;
        end
    While Y < 5 do
        E := E - B;
    end
end
```

Tabel Informasi

Dua fungsi penting Tabel Informasi

- Untuk membantu pemeriksaan kebenaran semantik dari program sumber
 - Untuk membantu dan mempermudah dalam pembuatan intermediate code dan proses pembuatan kode-kode (pembangkitan kode)
-

Tabel Informasi

Secara umum, sebuah tabel simbol bisa memiliki elemen-elemen tabel sebagai berikut, meskipun tidak semuanya dipergunakan oleh semua compiler

- No.urut identifier: menentukan nomor urut pada tabel simbol
 - Nama identifier
-

Tabel Informasi - Kegunaan



- Tipe identifier
- Object time address
- Dimensi dari identifier yang bersangkutan
- Nomor baris variabel yang dideklarasikan
- Nomor baris variabel yang direferensikan
- Field link

Tabel Informasi - Implementasi

Ada beberapa jenis Tabel Informasi

- **Tabel identifier**; berfungsi menampung semua identifier yang terdapat dalam program
 - **Tabel Array**: berfungsi menampung informasi tambahan untuk sebuah array
 - **Tabel blok**: mencatat variabel-variabel yang ada pada blok yang sama
 - **Tabel Real**: Menyimpan elemen tabel bernilai real
 - **Tabel string**: menyimpan informasi string
 - **Tabel display**: mencatat blok yang aktif
-

Tabel Informasi - Identifier

Tabel Identifier memiliki;

- No Urut identifier dalam tabel
 - Nama Identifier
 - Jenis dari identifier; seperti Prosedur, fungsi, tipe variabel dan konstanta
 - Tipe dari identifier yang bersangkutan; seperti Integer (bilangan bulat), Char, boolean , array, record, file
 - level dari identifier (depth of block); hal ini menyangkut letak identifier dalam program, konsepnya sama dengan pembentukan *tree*, misalnya main program level 0
-

Tabel Informasi - Identifier

Untuk **identifier**, pencatatan dapat berupa seperti;

- Alamat *relatif/address* dari identifier untuk implementasi
 - Informasi referensi dari identifier tertentu ke alamat tabel identifier yang lainnya
 - *link*; menghubungkan antar identifier
 - Normal: digunakan pada pemanggilan parameter, untuk membedakan parameter by value dan by reference
 - Contoh (dalam pascal)
-

Tabel Informasi - Identifier

```
Program A;  
Var B : Integer;  
  Procedure X (Z: char)  
    var C : Integer  
  begin  
    ....dst
```

Tabel identifier akan mencatat semua identifier;

0	A
1	B
2	X
3	Z
4	C

Tabel Informasi - contoh

Tabld: Array [0..tabmax] of record

nama : String;

link : integer;

Obj : object;

Tipe : Types;

ref: Integer;

normal : Boolean;

Level : 0.. Maxlevel;

address : Integer;

End

Dimana

objek =(konstant, variabel, prosedur, fungsi)

Types = (notipe, int, reals, booleans, chars, arrays, record

Tabel Informasi - Array

Tabel Array

dipergunakan untuk menyimpan informasi suatu identifier yang bertipe array, tabel ini memiliki field:

- No. Urut suatu array dalam tabel
 - Tipe dari indeks array yang bersangkutan
 - Tipe element array
 - Referensi dari elemen array
 - Index batas atas dan bawah array
 - Jumlah elemen array
 - Ukuran total array (total = atas - bawah + 1) x elemen size
 - Elemen size
-

Tabel Informasi - Block

Tabel Blok

Dipergunakan untuk menyimpan informasi blok-blok yang ada pada tabel utama. Berisikan field

- no urut blok
 - batas awal blok
 - batas akhir blok
 - ukuran parameter/parameter size
 - ukuran variabel/ variabel size
 - last variabel
 - last parameter
-

Tabel Informasi - Block

Contoh

Program A

Var B : Integer;

Procedure X (Z:char);

Var C : Integer;

begin

....

Untuk	Blok A	Blok B
last variable	= 2	4
Variable size	= 2 (dianggap int 2 byte)	2
Last parameter	= 0 (tanpa parameter)	3
parameter size	= 0	1 (char butuh 1 byte)

Tabel Informasi - Implementasi

Tabel Real

Dipergunakan untuk menyimpan nilai dari suatu identifier yang bertipe real (pecahan). Elemen-elemen dari tabel ini adalah sebagai berikut;

- NO urut elemen
- Nilai real suatu variabel real yang mengacu ke indeks tabel ini

Pemikirannya disini setiap tipe yang memiliki oleh suatu bahasa akan memiliki tabelnya sendiri

Tabel Informasi - Implementasi

Tabel String

Dipergunakan untuk menyimpan informasi string yang terdapat pada program sumber. Elemen-elemen yang terdapat dalam tabel ini adalah:

- ❑ no Urut elemen
- ❑ Karakter-karakter yang merupakan konstanta



Tabel Informasi - Implementasi

Tabel Display

menyimpan informasi-informasi mengenai blok-blok yang lagi aktif.

Elemen-elemen yang terdapat dalam tabel ini adalah:

- ❑ No Urut tabel
- ❑ Blok yang aktif

Pengisian tabel display dilakukan dengan konsep stack
